

EXPERIMENT 11: MAP COLORING USING (CSP)

AIM

To develop a Python program that solves the Map Coloring Problem using the Constraint Satisfaction Problem (CSP) approach by assigning different colors to adjacent regions.

OBJECTIVE

- To understand the concept of Constraint Satisfaction Problems (CSP) in Artificial Intelligence.
- To assign colors to map regions while satisfying the given constraints.
- To ensure that no two adjacent regions have the same color.
- To obtain a valid coloring solution using backtracking.

ALGORITHM

Step 1: Start.

Step 2: Define the map regions and their neighbouring regions.

Step 3: Define the available colors (e.g., Red, Green, Blue).

Step 4: Select an unassigned region.

Step 5: Assign a color that does not violate the coloring constraints.

Step 6: Check whether adjacent regions have different colors.

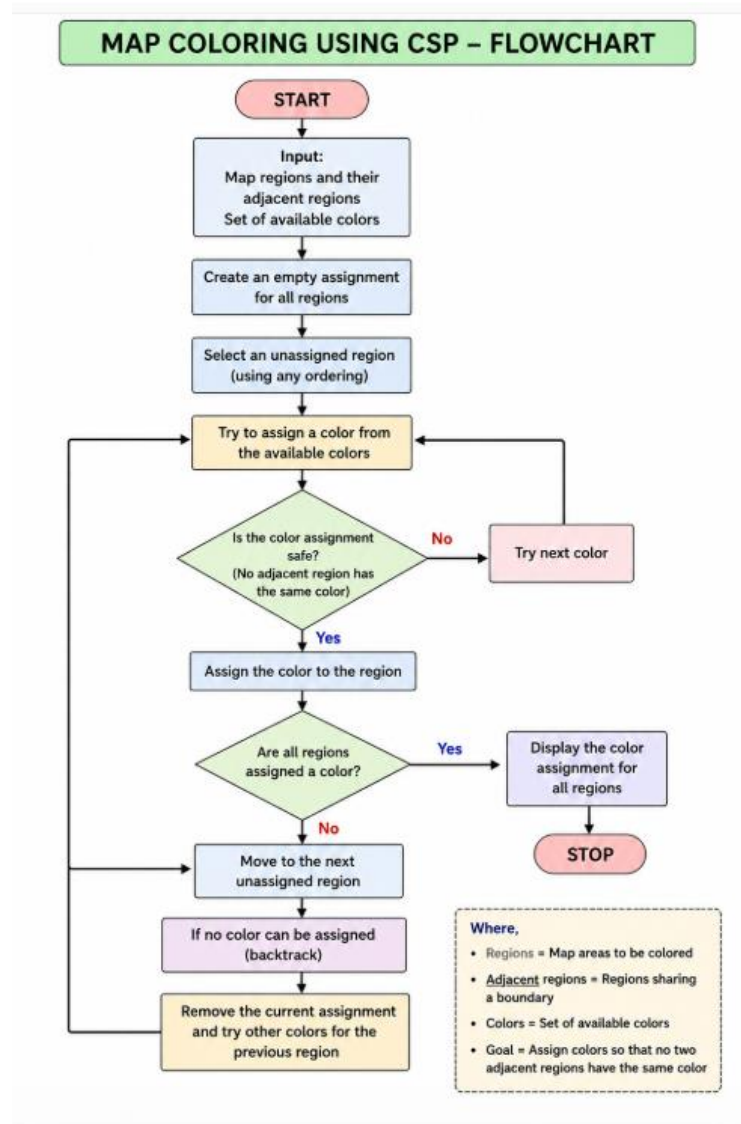
- If the assignment is valid, proceed to the next region.
- Otherwise, choose another color (Backtracking).

Step 7: Repeat Steps 4–6 until all regions are assigned valid colors.

Step 8: Display the color assigned to each region.

Step 9: Stop.

FLOW CHART



SOURCE CODE

Map Coloring using Constraint Satisfaction Problem (CSP)

Adjacency list of the map

```
graph = {  
    'A': ['B', 'C'],
```

```
'B': ['A', 'C', 'D'],  
'C': ['A', 'B', 'D', 'E'],  
'D': ['B', 'C', 'E'],  
'E': ['C', 'D']  
}
```

```
# Available colors
```

```
colors = ['Red', 'Green', 'Blue']
```

```
# Dictionary to store assigned colors
```

```
color_assignment = {}
```

```
# Check if assigning a color is valid
```

```
def is_safe(region, color):
```

```
    for neighbor in graph[region]:
```

```
        if neighbor in color_assignment and color_assignment[neighbor] == color:
```

```
            return False
```

```
    return True
```

```
# Backtracking function
```

```
def solve(region_index, regions):
```

```
    if region_index == len(regions):
```

```
        return True
```

```
    region = regions[region_index]
```

```
    for color in colors:
```

```
    if is_safe(region, color):
        color_assignment[region] = color

    if solve(region_index + 1, regions):
        return True

    # Backtrack
    del color_assignment[region]

return False

# List of regions
regions = list(graph.keys())

# Solve the CSP
if solve(0, regions):
    print("Map Coloring Solution:\n")
    for region in regions:
        print(region, "->", color_assignment[region])
else:
    print("No solution exists.")
```

OUTPUT

```
Map Coloring Solution:
A -> Red
B -> Green
C -> Blue
D -> Red
E -> Green
```

RESULT

The Python program successfully solves the Map Coloring Problem using the Constraint Satisfaction Problem (CSP) approach. It assigns colors to all regions such that no two adjacent regions have the same color, satisfying all the given constraints and producing a valid map coloring solution.